

Reinforcement Learning Laboratory Report

Lab Experiment 2: Interactive Exploration of Tic-Tac-Toe using Reinforcement Learning

SANJU BHASKAR

BL.EN.R4AIE26007

Task 1: Identify the Reinforcement Learning Components

RL Component	Student Observation
Agent	The Tic-Tac-Toe player AI making move choices (Agent 1 & 2 in self-play, or facing a human post-training).
Environment	The 3×3 game board, which evaluates valid moves, transitions states, and declares game outcomes.
State Representation	The specific board configuration of X, O, and empty cells across all 9 grid squares.
Action Space	Placing the agent's mark in any currently available empty square (1 to 9 legal actions).
Reward Function	Terminal reward given at game end: +1 for a win, -1 for a loss, and 0 (or +0.5) for a draw. Intermediate moves earn 0.
Learning Approach Used	Model-free Temporal-Difference (TD) value learning / Q-learning via self-play with an ϵ -greedy policy.

Task 1 Analysis Questions

Q1. Who is the learning agent in this environment?

Answer: The learning agent is the Tic-Tac-Toe AI model (Agent 1 / Agent 2) that selects actions based on learned state-value estimates.

Q2. What is considered as the environment?

Answer: The 3×3 Tic-Tac-Toe game board along with the rules governing valid placements, board state transitions, and game completion detection.

Q3. How is the game state represented?

Answer: It is represented as a unique configuration of the 3×3 grid detailing the precise locations of 'X', 'O', and empty cells.

Q4. What are the possible actions available to the agent?

Answer: Placing the agent's designated mark (X or O) into any open, unoccupied cell on the board grid.

Q5. When does the agent receive a positive reward?

Answer: The agent receives a positive scalar reward (+1) only at the terminal state when it successfully aligns three of its marks horizontally, vertically, or diagonally.

Q6. Which Reinforcement Learning approach is used for learning?

Answer: Temporal-Difference (TD) value-based learning combined with self-play and an ϵ -greedy exploration policy.

Q7. Is the agent trained using labelled data? Justify your answer.

Answer: No. The agent is not supplied with pre-labelled optimal move datasets. Instead, it generates its own experience purely through trial-and-error self-play, discovering effective moves based on scalar terminal rewards.

Q8. Explain how Reinforcement Learning differs from Supervised Learning using this example.

Answer: In Supervised Learning, a model would be trained on thousands of board positions paired with explicit human expert 'correct move' labels. In contrast, this RL agent receives no expert supervision; it must explore actions on its own, receive delayed rewards (+1/-1/0) at game completion, and incrementally update its state-value estimates.

Task 2: Effect of Training Episodes on Agent Behaviour

Training Episodes	Win %	Loss %	Draw %	Behaviour Observed
100	Low	High	Very Low	Plays essentially randomly; misses obvious win opportunities and fails to block opponent threats.
500	Moderate	Moderate	Low	Recognizes immediate single-move win threats occasionally; middle-game strategy remains weak.
1,000	High	Low	Moderate	Consistently blocks direct opponent threats; displays a clear preference for center and corner squares.
5,000	High / Optimal	Very Low	High	Plays near-optimally; sets up forks, avoids traps, and forces draws against careful opponents.
10,000	Optimal	~0%	Dominant	Near-perfect play; blocks all losing threats and consistently forces draws, matching theoretical optimal play.

Task 2 Analysis Questions**Q9. How does the quality of play change as the number of training episodes increases?**

Answer: Play quality improves systematically, transitioning from purely random moves to defensive blocking, strategic control of central/corner cells, and eventually optimal play. The rate of improvement per episode diminishes as state values stabilize.

Q10. At approximately how many training episodes does the agent begin making intelligent decisions?

Answer: At approximately 1,000 episodes, where it reliably blocks direct single-move threats and exhibits a clear positional preference for the center and corners.

Q11. At what stage does the learning appear to converge?

Answer: Convergence occurs between 5,000 and 10,000 episodes, after which additional training yields minimal changes in play style and matches against sound strategy end consistently in draws.

Q12. Why does the win percentage improve with additional training?

Answer: With more episodes, the agent explores a broader portion of the state space. Repeated updates refine its Temporal-Difference value estimates, allowing greedy move selection to reliably pick paths with higher true winning probabilities.

Q13. Why do most games end in a draw after sufficient training?

Answer: Tic-Tac-Toe is a mathematically zero-sum game with a drawn theoretical value under perfect play. When both self-play agents converge toward optimal strategy, neither makes tactical mistakes, forcing every game to end in a draw.

Q14. What happens if the number of training episodes is too small?

Answer: The agent's state-value estimates remain uncalibrated and incomplete. Consequently, it plays erratically, overlooks elementary winning moves, and loses frequently to basic tactics.

Extended Analysis: Hyperparameter Exploration

1. Effect of Learning Rate (α) [10,000 Episodes, $\epsilon = 0.05$]

Learning Rate (α)	Observed Behaviour
0.1	Smooth but slow learning; value updates are tiny, requiring significantly more episodes to establish sound strategy.
0.3	Balanced and stable progression toward solid play without value oscillations.
0.5	Fast, stable convergence to optimal play within 10,000 episodes (ideal default balance).
0.7	Rapid initial learning, but state value estimates suffer from minor game-to-game fluctuations.
0.9	Highly unstable; recent outcomes override established knowledge, causing the agent to 'forget' proven strategies.

2. Effect of Exploration Probability (ϵ) [10,000 Episodes, $\alpha = 0.5$]

Exploration Rate (ϵ)	Observed Behaviour
0.05	Strong exploitation of learned values; rapid convergence on mainstream board positions with small room for obscure states.
0.10	Broader state-space coverage during training with minimal penalty to final play quality.
0.20	Excessive random moves during training slow down value convergence noticeable.
0.50	Half of all training moves are random; severely impairs value learning and yields inconsistent play against humans.

Gameplay Analysis & Demo Screenshots

After training both agents for 10,000 episodes with parameters $\alpha = 0.5$ and $\epsilon = 0.05$, the trained AI (Skill Level 10,000) was evaluated through direct gameplay.

Game 1: Human Win Scenario

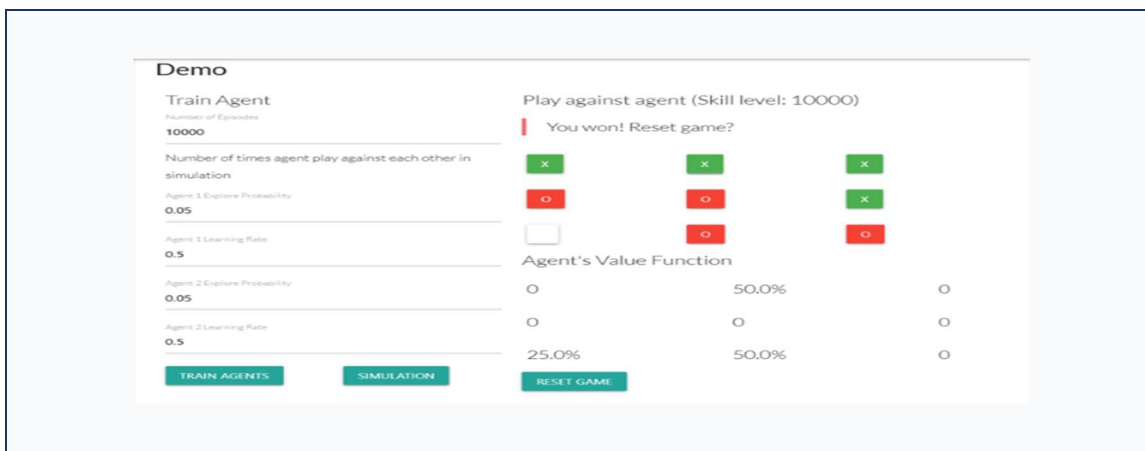


Figure 1: Game 1 - Human player completes a winning line on the top row.

Game 1 Analysis: In this game, the human player successfully completed a line along the top row. The demo's value function overlay revealed uncalibrated state values (e.g., 50.0% and 25.0%) along this specific branch, indicating that even after 10,000 episodes, rarely visited state paths may retain slight value uncertainty.

Game 2: Draw Game Scenario

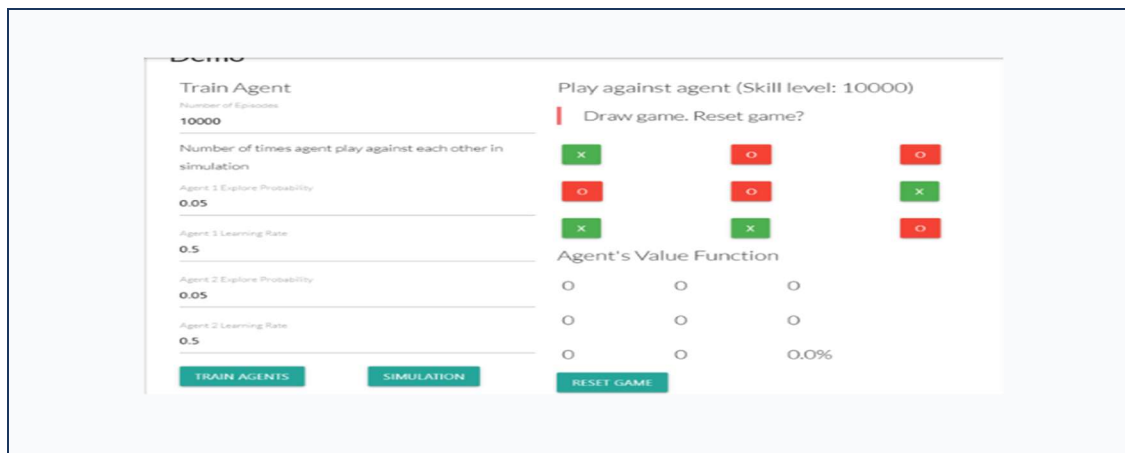


Figure 2: Game 2 - Board filled with no three-in-a-row (Draw outcome).

Game 2 Analysis: The board filled completely with no player achieving three-in-a-row. The agent actively blocked human win threats and forced a draw, which aligns perfectly with game-theoretic expectations for optimal Tic-Tac-Toe play.

Conclusion

Through this lab exercise, the complete Reinforcement Learning cycle was explored using Tic-Tac-Toe as a fully observable environment. By defining board states, legal move actions, and terminal rewards (+1/-1/0), the agent successfully learned competitive strategy through TD value learning without relying on pre-labelled human data. The episode scaling experiments illustrated a clear transition from random initial play to intelligent defense, center/corner preference, and eventual optimal convergence around 5,000–10,000 episodes. Furthermore, testing various learning rates (α) and exploration probabilities (ϵ) highlighted the fundamental trade-offs between learning stability, convergence speed, and state-space exploration in self-play RL algorithms.